Documentação da Sprint 2 — Bit or Byte

Projeto: BorB — Plataforma de Estudos Scrum

Período da Sprint 2: 23/10/2026 – 05/11/2026

Curso / Instituição: FATEC Jacareí – 1º DSM

Descrição do Trabalho

É comum que estudantes e profissionais iniciantes em TI encontrem dificuldades para fixar conceitos teóricos de metodologias ágeis (como Scrum e Kanban) e aplicá-los em cenários reais de desenvolvimento. A falta de ferramentas práticas e interativas para mensurar o aprendizado gera uma lacuna no processo de formação. O **BorB — Plataforma de Estudos Scrum** visa resolver esse problema oferecendo uma trilha de simulados randômicos e cronometrados, acompanhada de uma Área de Estudos completa com 12 temas fundamentais, validando o conhecimento adquirido por meio da emissão de um certificado digital autêntico.

1. Equipe e Organização do Trabalho

1.1. Divisão de Papéis e Responsabilidades

- **Product Owner (PO):** Gestão e priorização do Product Backlog, organização do Kanban no GitHub Projects e integração final da branch `main`.
- **Scrum Master (SM):** Organização das demandas, facilitação das cerimônias do Scrum, garantia do progresso do Burndown e acompanhamento do time de desenvolvimento.
- **Equipe de Desenvolvimento (Dev Team):**
    - **Melhorias no Protótipo (Figma):** Nikolas Gabriel, Ryan Cesar.
    - **Documentação e Orientação da Sprint:** Sarah Cristiny.
    - **Atualizações no Readme:** Sarah Cristiny, Luiz Felipe, Nikolas Gabriel.
    - **Organização dos Temas de Estudo e Questões:** João Vitor Maximiano, Willian de Paula, Luiz Felipe, Ryan César, Sarah Cristiny e Matheus Oliveira.
    - **Banco de Dados (PostgreSQL):** Matheus Oliveira, Pedro Lucas.
    - **Manutenção do Código e Adição de Funcionalidades:** Matheus Oliveira, Pedro Lucas.

1.2. Gestão do Backlog e Kanban

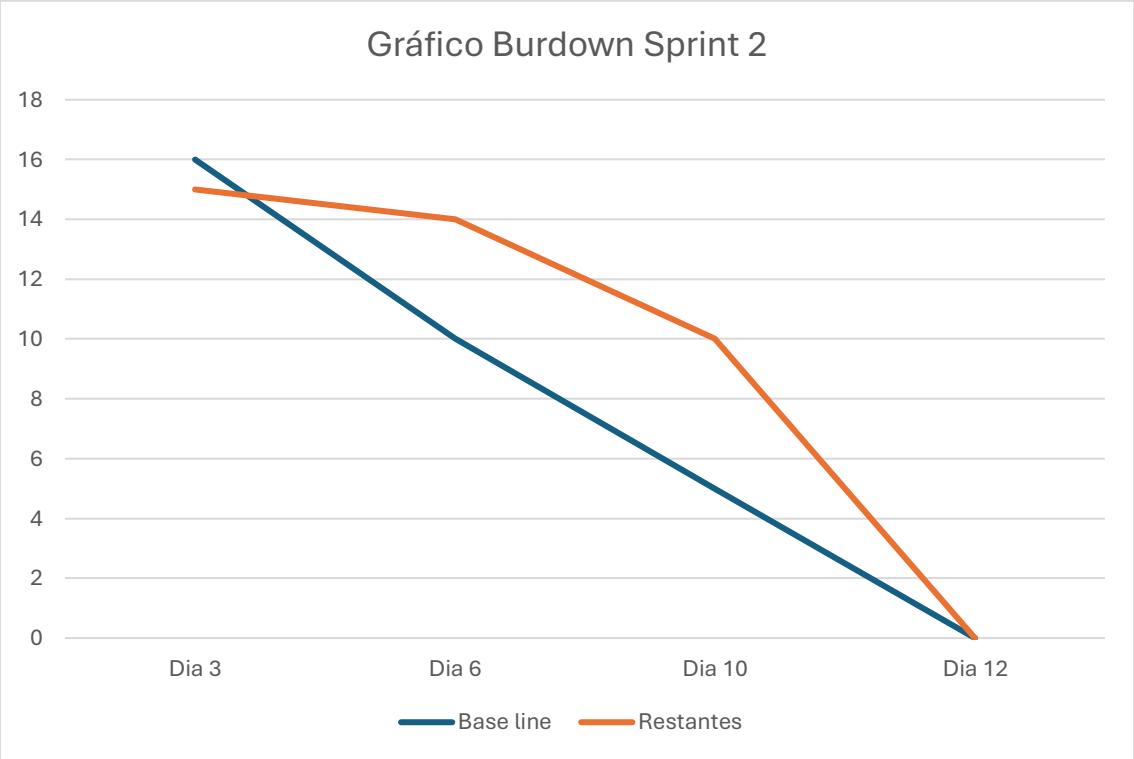
- O fluxo de trabalho foi gerenciado via GitHub Projects.
- Na Sprint Planning, foi definida uma meta inicial de 15 tarefas.
- Ao longo da Sprint, o escopo foi reajustado para 16 tarefas para acomodar novas necessidades e melhorias arquiteturais.

2. Funcionalidades Desenvolvidas e Implementadas (RNF06)

1. **Mapeamento e Povoamento do Banco de Dados Relacional**
    - **Implementação:** Estruturação e povoamento do banco relacional PostgreSQL via scripts DDL/DML e migrações. Armazenamento padronizado de 12 temas, questões contendo enunciado, suporte a imagens ilustrativas, 4 alternativas por questão e vinculação da resposta correta.
    - **Requisitos Atendidos:** RF06, RF08, RP04, RP06.
2. **Área de Estudos e Visualização de Temas (Front-end)**
    - **Implementação:** Desenvolvimento da interface da Área de Estudos em HTML5, CSS3 e JavaScript puro, permitindo ao candidato visualizar os 12 temas da certificação, acessar o conteúdo didaticamente estruturado e acompanhar quais temas ainda estão pendentes de realização.
    - **Requisitos Atendidos:** RF05, RF07, RF21, RP01.
3. **Início, Sorteio Randômico e Resolução da Questão**
    - **Implementação:** Criação dos endpoints em Node.js responsáveis por permitir ao candidato iniciar um tema pendente e sortear aleatoriamente 1 questão do tema selecionado. Implementação da mecânica de seleção e envio da alternativa pela interface para registro no servidor.
    - **Requisitos Atendidos:** RF05, RF06, RF08.
4. **Feedback de Correção Imediato**
    - **Implementação:** Exibição imediata da resposta correta e validação do gabarito logo após o envio da alternativa pela pessoa candidata, permitindo a verificação de acerto ou erro antes do avanço na certificação.
    - **Requisitos Atendidos:** RF09.
5. **Navegação, Continuidade e Fluxo de Sessão**
    - **Implementação:** Interface e regras de negócio que permitem ao candidato decidir, ao final de cada questão respondida, se deseja avançar imediatamente para o próximo tema pendente ou encerrar a sessão com segurança para retomar a prova posteriormente.
    - **Requisitos Atendidos:** RF12, RF13.
6. **Controle de Tempo e Tratamento de Exceções da Prova**
    - **Implementação:** Implementação de cronômetro regressivo visível de 150 segundos em tempo real. Regra de encerramento automático da tentativa com perda de pontuação caso o tempo expire ou ocorra interrupção de conexão/fechamento de aba. Mecanismo de persistência que permite ao candidato retomar a certificação de onde parou para responder aos temas pendentes.
    - **Requisitos Atendidos:** RF10, RF11, RF13, RF14, RF15.
7. **Reestruturação Visual da Tela de Questões (UI/UX)**
    - **Implementação:** Reestruturação da tela de realização da prova no front-end para garantir uma experiência de usuário (UX) clara e organizada, com layout legível para o enunciado, exibição centralizada da imagem associada, campo de seleção de alternativas bem delimitado e botão de confirmação.
    - **Requisitos Atendidos:** RF08, RNF01.

8. **Documentação UML e Diagramas Técnicos**
    - **Implementação:** Elaboração da documentação técnica formal em UML (Diagramas de Casos de Uso, Diagrama de Classes e Diagrama de Sequência) cobrindo os fluxos de candidatos, autenticação, sorteio de questões e resolução do exame.
    - **Requisitos Atendidos:** RNF06.
9. **Vídeo de Demonstração do Incremento da Sprint 2**
    - **Implementação:** Gravada e publicada a demonstração audiovisual completa das funcionalidades da Sprint 2 rodando na plataforma BorB para apresentação e validação acadêmica.
    - **Requisitos Atendidos:** RNF05.

3. Gráfico Burndown e Acompanhamento



- **Dia 0:** Planejamento com 16 tarefas ajustadas.
- **Dia 3:** 2 tarefas concluídas e movidas no Kanban.
- **Dia 6:** +1 tarefa concluída (total de 3 entregas).
- **Dia 10:** O Product Owner reajustou a organização no Kanban, totalizando 7 tarefas concluídas. As demais tarefas estavam em estágio avançado.
- **Dia 12:** As tarefas restantes foram finalizadas com sucesso antes do término da Sprint, impulsionadas pelo fracionamento em subtarefas dentro de cada User Story.

4. Eventos e Práticas Scrum (RNF05)

- **Daily Scrum:** Realizada de forma remota via WhatsApp, onde o Scrum Master e o Product Owner monitoravam o progresso, alinhavam impedimentos e redistribuíam frentes de trabalho.
- **Critério de Pronto (Definition of Done — DoD):** Validação funcional, código devidamente integrado e aprovado nas branches correspondentes.

4.1. Registro Oficial de Daily Scrums — Sprint 2

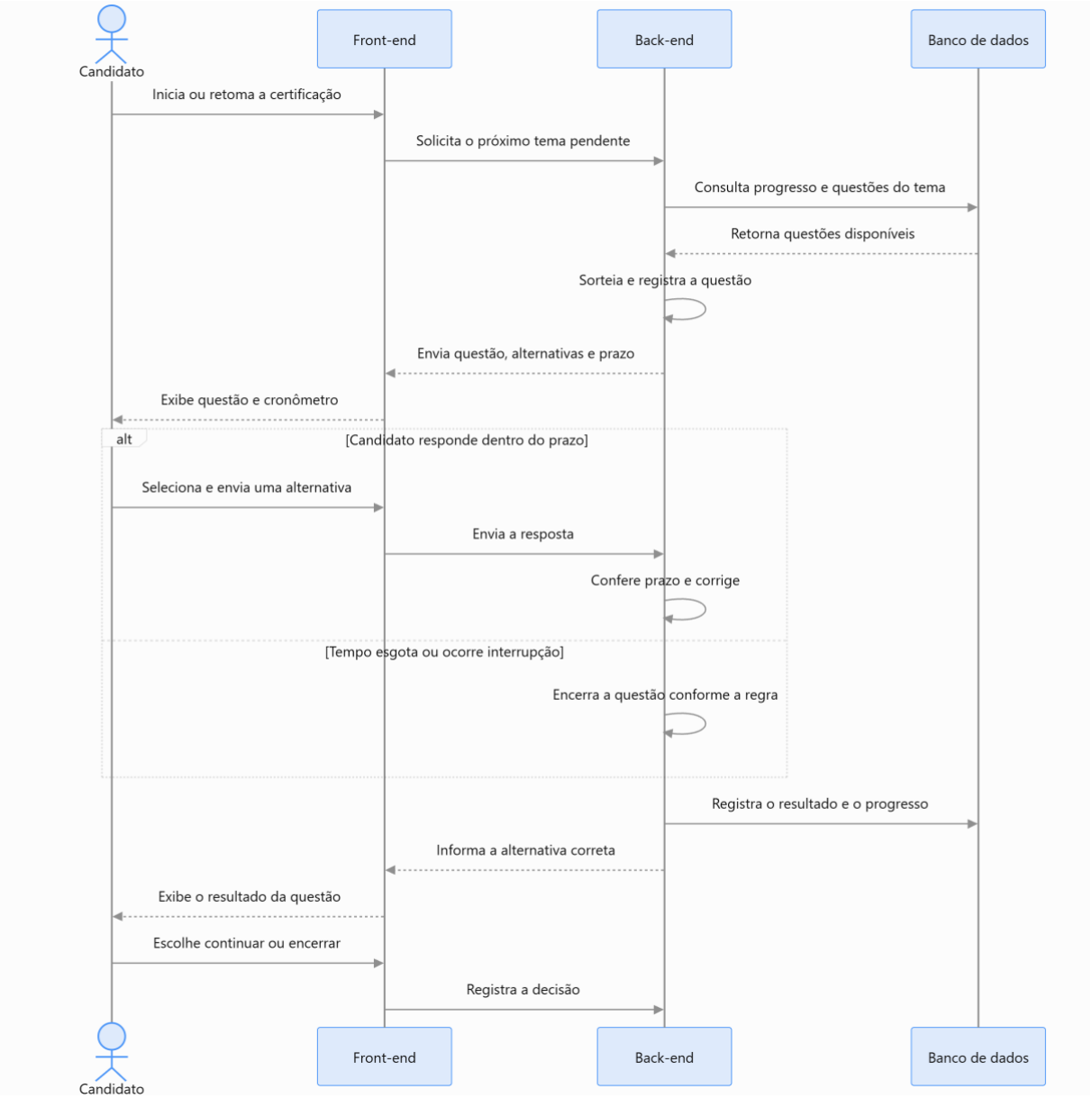
- **Daily Scrum — 26/10/2026 (Segunda-feira)**
  - *Foco da Iteração:* Entrega da parte funcional dos temas, questões, exame e progresso de certificação com histórico.
  - *O que foi discutido:* Divisão de conteúdo entre os membros para aprofundamento dos 12 temas; direcionamento de Matheus e Pedro para a modelagem UML no Astah; alinhamento secundário da equipe Figma.
- **Daily Scrum — 27/10/2026 (Terça-feira)**
  - *O que foi discutido:* Início da criação da estrutura de dados para o armazenamento das questões sorteadas e base inicial dos cronômetros no front-end. Alinhamento sobre a necessidade de estabilizar o banco antes de plugar a lógica de sorteio.
- **Daily Scrum — 28/10/2026 (Quarta-feira)**
  - *O que foi discutido:* Foco total de Matheus e Pedro no banco e navegação; demandas direcionadas à equipe Figma para tornar o protótipo navegável e cobrir telas de erro/tempo esgotado; prioridade máxima da equipe front-end no vínculo dos temas e questões.
- **Daily Scrum — 29/10/2026 (Quinta-feira)**
  - *O que foi discutido:* Verificação do andamento da modelagem no Astah e refinamento de conteúdos pela equipe de front-end, reforçando o padrão de qualidade necessário para as questões.
- **Daily Scrum — 30/10/2026 (Sexta-feira)**
  - *O que foi discutido:* Validação do fluxo de navegação das questões e verificação de commits associados no repositório; monitoramento das tabelas de alternativas e feedback individual para evitar estagnação no fim de semana.
- **Daily Scrum — 03/11/2026 (Segunda-feira)**
  - *Foco da Iteração:* Reta final da sprint. Cobrança de status detalhado por User Story (US11 até US23) para assegurar o fechamento das pendências.
- **Daily Scrum — 04/11/2026 (Terça-feira) — Encerramento / Alinhamento Crítico**
  - *Foco da Iteração:* Levantamento dos últimos débitos técnicos e inconsistências combatidos na reta final (ajuste nas imagens das questões, normalização do volume de questões por tema, consolidação dos scripts Docker, unificação do fix de rolagem do cronômetro na branch `develop`, e refinamento dos botões de encerramento de sessão).

5. Versionamento e Engenharia de Software (Git & GitHub)

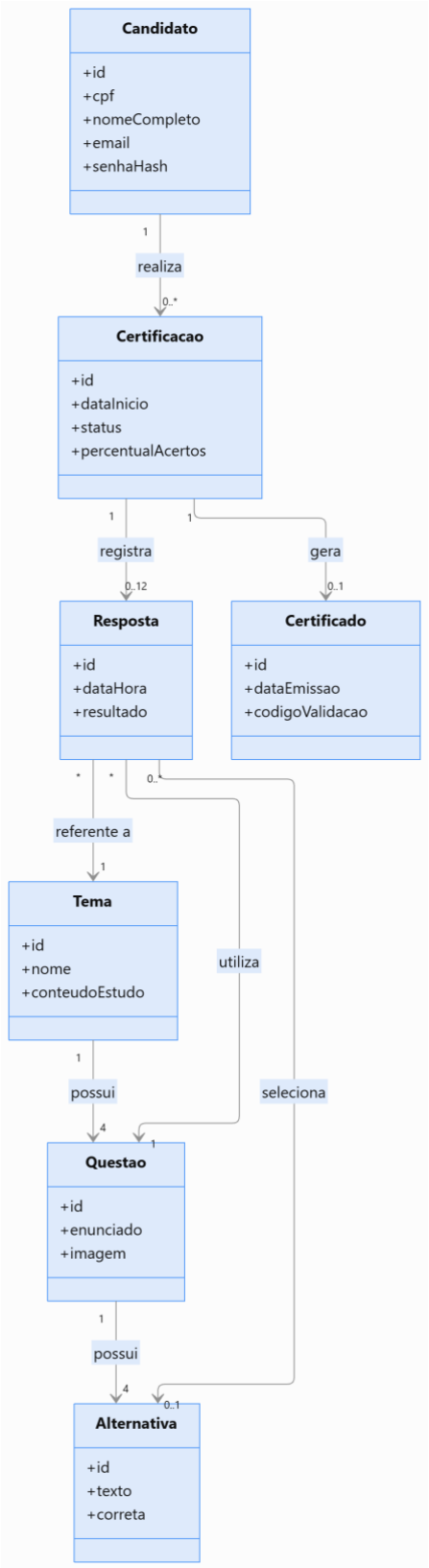
- **Estratégia de Branches:** Os desenvolvedores atuaram em branches isoladas por funcionalidade, integrando de forma contínua na branch `develop`.
- **Consolidação:** Ao término da Sprint 2, após os testes e code reviews, foi efetuado o merge da `develop` para a `main`.

6. Diagramas de Casos de Uso, Classes e Sequência

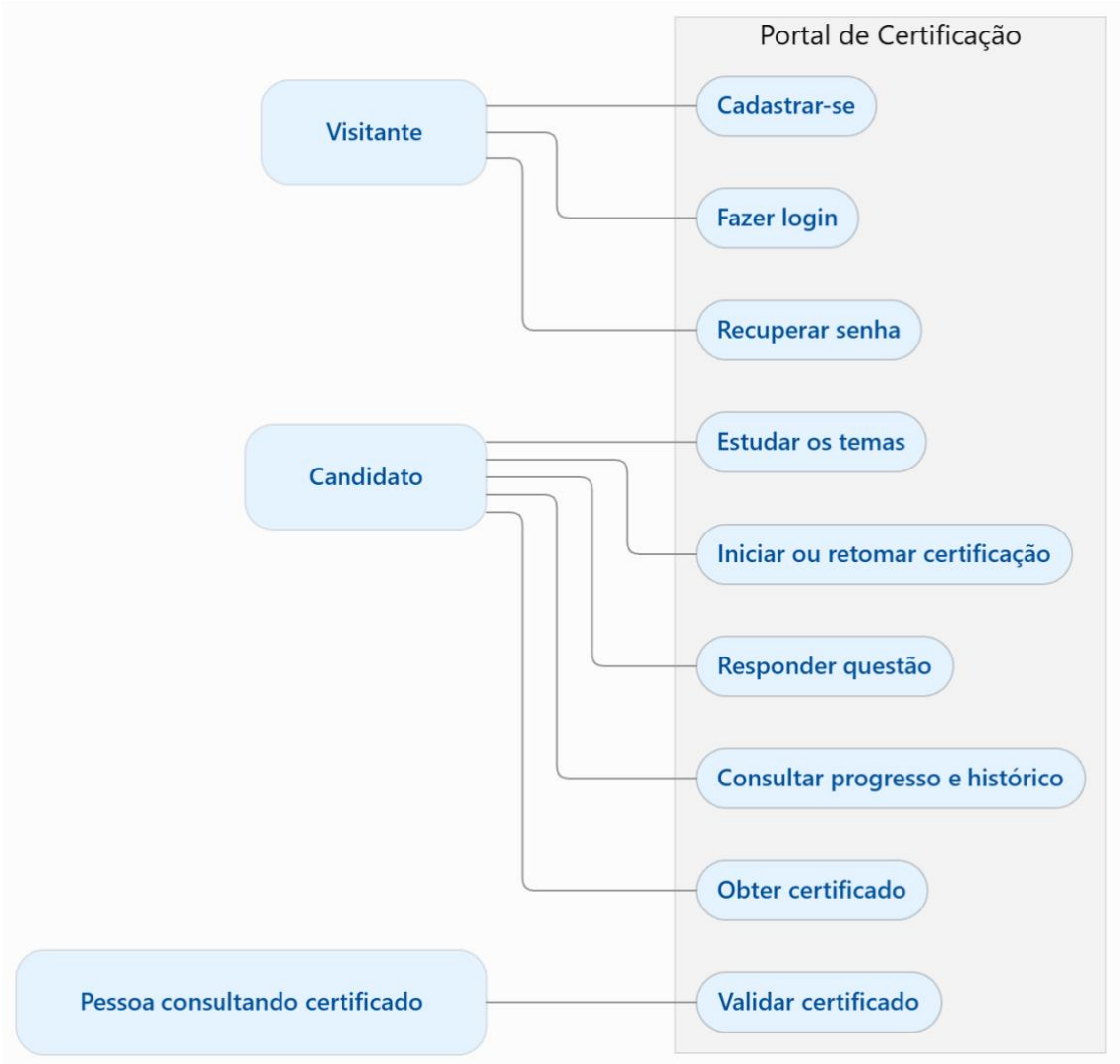
- **Diagrama de Casos de Uso**



- **Diagrama de Classes**



- **Diagrama de Sequência**



7. Documentação das APIs (Endpoints — RNF06)

7.1. Autenticação e Usuários

- **POST /api/auth/register**
  - *Descrição:* Realiza o cadastro de um novo candidato.
  - *Body (JSON):* `{"nome": "Nome", "cpf": "000.000.000-00", "email": "email@email.com", "senha": "senha"}`
  - *Respostas:* `201 Created` / `400 Bad Request`
- **POST /api/auth/login**
  - *Descrição:* Autentica o usuário por CPF e senha.
  - *Body (JSON):* `{"cpf": "000.000.000-00", "senha": "senha"}`

    - *Respostas:* `200 OK` (retorna cookie/token de sessão) / `401 Unauthorized`
- **POST /api/auth/forgot-password**
    - *Descrição:* Solicita recuperação de senha (modo console: código enviado diretamente aos logs do contêiner Docker).
    - *Body (JSON):* `{"email": "email@email.com"}`
    - *Respostas:* `200 OK` / `404 Not Found`

8. Instruções de Instalação e Execução (RNF06)

Pré-requisitos

- Git instalado.
- Docker Desktop em execução (com Docker Compose integrado). *(Não é necessário instalar Node.js ou PostgreSQL localmente).*

Passos para Execução

1. Clonar o Repositório:
2. `git clone https://github.com/bit-or-byte-devs/ABP-1-Portal-de-Certificacao-Scrum.git`
3. `cd ABP-1-Portal-de-Certificacao-Scrum`
4. Subir os Contêineres:
5. `docker compose up -d --build`

   (Sobe os serviços `db` — PostgreSQL 16 com carga automática de esquema/temas — e `app` — API Node.js e servidor front-end).

6. Acessar a Aplicação: Abra o navegador em `http://localhost:3000/pages/index.html`

Comandos Úteis

- `docker compose logs -f app` — Acompanhar logs e códigos de recuperação de senha.
- `docker compose down` — Parar serviços mantendo os dados.
- `docker compose down -v` — Parar e resetar o banco de dados do zero.

9. Sprint Retrospective (Retrospectiva da Sprint 2)

9.1. O que funcionou bem? (Good Points)

- **Comunicação e Monitoramento Remoto:** Uso ágil do WhatsApp para as Dailies e alinhamento contínuo, facilitando a tomada de decisões e o suporte mútuo entre as frentes.
- **Evolução Técnica e Arquitetural:** Sucesso na estruturação completa do banco relacional PostgreSQL (com scripts DDL/DML), na lógica de sorteio randômico em Node.js e no tratamento de exceções (incluindo o cronômetro regressivo de 150 segundos e persistência de sessão).

- **Entrega de Requisitos Críticos (RF08 e RP06):** Superação do débito técnico inicial de imagens, com a reestruturação visual da UI/UX da tela de questões para garantir a exibição centralizada de imagens ilustrativas e suporte total aos requisitos visuais e funcionais.
- **Engajamento na Reta Final:** Esforço coletivo expressivo nos últimos dias para zerar as pendências do Kanban e consolidar a documentação UML (RNF06) junto ao vídeo de demonstração do incremento.
- **Rastreabilidade:** Clareza na definição e execução das User Stories (US11 a US23).

9.2. O que não funcionou? (Bad Points)

- **Configuração de Ambiente (Docker):** Comportamento inicial onde o Docker subia sem popular automaticamente a carga inicial de dados em cenários limpos, exigindo atenção extra na configuração dos scripts de inicialização.
- **Validações de Banco de Dados:** Dependência inicial excessiva da camada de aplicação para validações que deveriam ser garantidas de forma nativa por restrições (`constraints`) no banco de dados relacional.
- **Sobrecarga na Reta Final:** Concentração de ajustes visuais e refinamentos de UI/UX próximo ao prazo de encerramento da sprint, gerando desgaste desnecessário.

9.3. DoR e DoD da Sprint 2

DoR da Sprint 2 (Definition of Ready)

- **Modelagem de Dados Aprovada:** Modelagem do banco de dados validada (PostgreSQL) para suportar os 12 temas, 48 questões (4 por tema), alternativas, imagens ilustrativas, tentativas únicas por tema e histórico de interrupções.
- **Carga e Conteúdo Homologados:** Definição final dos 12 temas da certificação e estruturação dos arquivos JSON/scripts de migração contendo o conteúdo didático e as questões com imagens.
- **Protótipos de Alta Fidelidade (Figma):** Telas aprovadas pela equipe para a Área de Estudos, Fluxo da Prova (com cronômetro, enunciado, imagem e alternativas conforme especificações) e fluxo de interrupção/retomada.
- **Especificação de Regras de Negócio:** Critérios de aceite detalhados para o sorteio randômico de questões (US13), controle do cronômetro regressivo (US20), encerramento por tempo esgotado (US21) e tratamento de interrupções de sessão (US22/US23).

DoD da Sprint 2 (Definition of Done)

- **Banco de Dados e Povoamento:** Schema DDL e seeds DML executados via Docker Compose, com carga íntegra dos 12 temas e das 48 questões (4 por tema) e migrações (001 a 005) testadas e integradas.
- **Núcleo de Back-end (Motor do Simulado & Regras):** Endpoints implementados em Node.js para sorteio aleatório (US13), tentativa única (US16, US19), controle de cronômetro, tempo esgotado (US21) e tratamento/retomada de interrupções (US22, US23).

- **Área de Estudos & Interface (Front-end):** Telas desenvolvidas em HTML5, CSS3 e JavaScript puro, estruturadas nos 12 temas (US11, US15) e com reestruturação da tela de prova para enunciado, imagens e alternativas.
- **Feedback e Diagramas UML:** Exibição da resposta correta após submissão (US18) e diagramas UML de Classes, Casos de Uso e Sequência documentados.

9.4. Plano de Action Items para a Sprint 3

- **Antecipação e Automatização da Carga de Dados (Docker):** Garantir que os scripts DML de povoamento e os arquivos JSON de temas rodem de forma automatizada no `docker compose up`, evitando que o ambiente suba limpo ou sem as questões essenciais.
- **Rigidez nas Regras de Banco de Dados:** Implementar restrições nativas (`constraints`) no PostgreSQL para obrigar exatamente 4 alternativas e 1 resposta correta por questão, reduzindo a dependência de validações isoladas no front-end.
- **Gestão Ágil de Branches (Merges Diários):** Evitar que correções importantes fiquem presas em branches isoladas (como o ajuste do cronômetro), estabelecendo rotinas diárias de unificação na branch `develop`.

10. Documentação dos Temas

Documento com os temas dos estudos

11. Link do Protótipo (Figma)

Figma

12. Vídeo de Apresentação (YouTube)

Vídeo da Sprint 2

13. Documentação Complementar e Referências

- **Instruções de Importação de Questões:** `docs/carga-questoes.md`
- **Mapeamento Curricular de Temas:** `docs/questoes/mapeamento-temas.md`
- **Relatório de Integração:** `docs/relatorio-integracao-2026-11-02.md`
- **Instruções de Importação de Conteúdos:** `docs/carga-temas.md`